

Smart Thermal Control System

Task-4 — IoT Cloud Integration & Real-Time Remote Monitoring

Name: Rajesh Lingam

B.Tech in ECE

Mohan Babu University, Tirupati

Cloud Dashboard: <https://thingspeak.mathworks.com/channels/3423267>

Simulation Link: <https://wokwi.com/projects/468880379614914561>

1 Objective

The objective of Task-4 is to take the Smart Automation System built in Task-3 and turn it into a proper cloud-connected IoT device — one that sends live sensor data to the internet so it can be monitored from anywhere, not just from the Serial Monitor sitting in front of you.

By completing this task, the following things were learned and implemented:

- How embedded devices actually talk to cloud platforms over WiFi
- How to publish live sensor readings to an online dashboard
- How real-world IoT dashboards are built and what data they show
- Why this skill matters — Smart Home, Industry 4.0, and automation systems all rely on exactly this kind of device-to-cloud pipeline

2 System Architecture

The diagram below shows how all the parts of this project are connected and how data moves through the system — starting from the sensor, through the ESP32, out to the local hardware, and finally up to the ThingSpeak cloud dashboard.

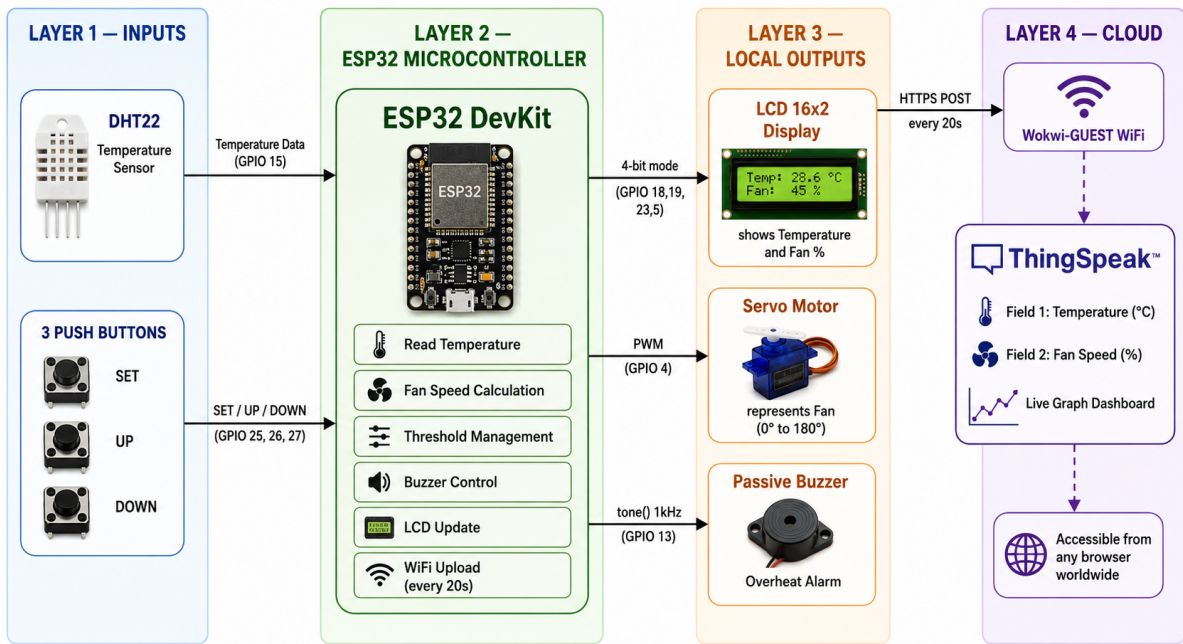


Figure 1: System Architecture — End-to-End Data Flow

Here is a quick walkthrough of each layer:

1. **Inputs** — The DHT22 sensor sends temperature readings to the ESP32 every 2 seconds through GPIO 15. The three push buttons (SET, UP, DOWN) connected to GPIO 25, 26, and 27 let the user change the low and high temperature thresholds without touching the code.
2. **ESP32 — The Brain** — This is where everything happens. The ESP32 reads the sensor, calculates the fan speed, decides whether the buzzer should be on or off, updates the LCD, and handles the WiFi upload every 20 seconds. All of this runs in one continuous loop.
3. **Local Outputs** — Three things respond to what the ESP32 decides. The LCD shows the current temperature and fan speed percentage in real time. The servo motor represents the fan — it moves from 0° (fan off) to 180° (full speed) proportionally based on temperature. The passive buzzer sounds a 1 kHz alarm through `tone()` when the temperature crosses the high threshold.
4. **Cloud** — Every 20 seconds, the ESP32 sends two values to ThingSpeak over WiFi using an HTTPS POST request. Field 1 receives the temperature in °C and Field 2 receives the fan speed as a percentage. ThingSpeak stores each reading with a timestamp and displays both as live line graphs on the dashboard — visible from any browser, anywhere in the world.

3 IoT Cloud Platform — ThingSpeak

Out of the available options (ThingSpeak, Blynk, Arduino IoT Cloud, Adafruit IO), **ThingSpeak by MathWorks** was chosen for this project. It is the most beginner-friendly option and works perfectly with the ESP32 without needing any extra app or paid plan.

Platform	Difficulty	Best For
ThingSpeak	Easy	Beginners (chosen)
Blynk Cloud	Easy + App	Attractive mobile dashboards
Arduino IoT Cloud	Medium	Arduino users
Adafruit IO	Medium	Data visualization

ThingSpeak automatically generates a graph for every field you send data to. No manual dashboard setup needed — you just send your readings and the charts appear on their own.

4 Hardware and Simulator

Since physical hardware was not used, the entire circuit was built and simulated on **Wokwi** — a browser-based electronics simulator that fully supports the ESP32, including its WiFi capability. This means the device was able to connect to a real WiFi network inside the simulation and actually upload data to ThingSpeak live.

Components Used

- ESP32 DevKit C V4 — the main microcontroller with built-in WiFi
- DHT22 — temperature and humidity sensor
- LCD 16x2 — displays temperature and fan speed locally
- Servo Motor — represents the fan (0° = OFF, 180° = Full Speed)
- Passive Buzzer — alarm when temperature crosses the high threshold
- 3 Push Buttons — SET, UP, DOWN for adjusting thresholds
- Potentiometer — controls LCD contrast

Circuit Connections

The table below lists every wire connection in the simulation exactly as it was built.

Component Pin	ESP32 Pin	Purpose
dht1:VCC	3V3	Sensor power
dht1:SDA	GPIO 15	Temperature data
dht1:GND	GND	Ground
lcd1:VSS	GND	LCD ground
lcd1:VDD	5V	LCD power
lcd1:V0	pot1:SIG	Contrast control
lcd1:RS	GPIO 21	Register select
lcd1:RW	GND	Write mode
lcd1:E	GPIO 22	Enable
lcd1:D4	GPIO 18	Data bit 4
lcd1:D5	GPIO 19	Data bit 5
lcd1:D6	GPIO 23	Data bit 6
lcd1:D7	GPIO 5	Data bit 7
lcd1:A	3V3	Backlight positive
lcd1:K	GND	Backlight negative
servo1:PWM	GPIO 4	Fan speed signal
servo1:V+	5V	Servo power
servo1:GND	GND	Ground
bz1:1	GPIO 13	Buzzer signal
bz1:2	GND	Ground
btn3:1.l	GPIO 25	SET button
btn2:1.l	GPIO 26	UP button
btn1:1.l	GPIO 27	DOWN button
btn3/2/1:1.r	GND	Button ground (all three)

Circuit Screenshot

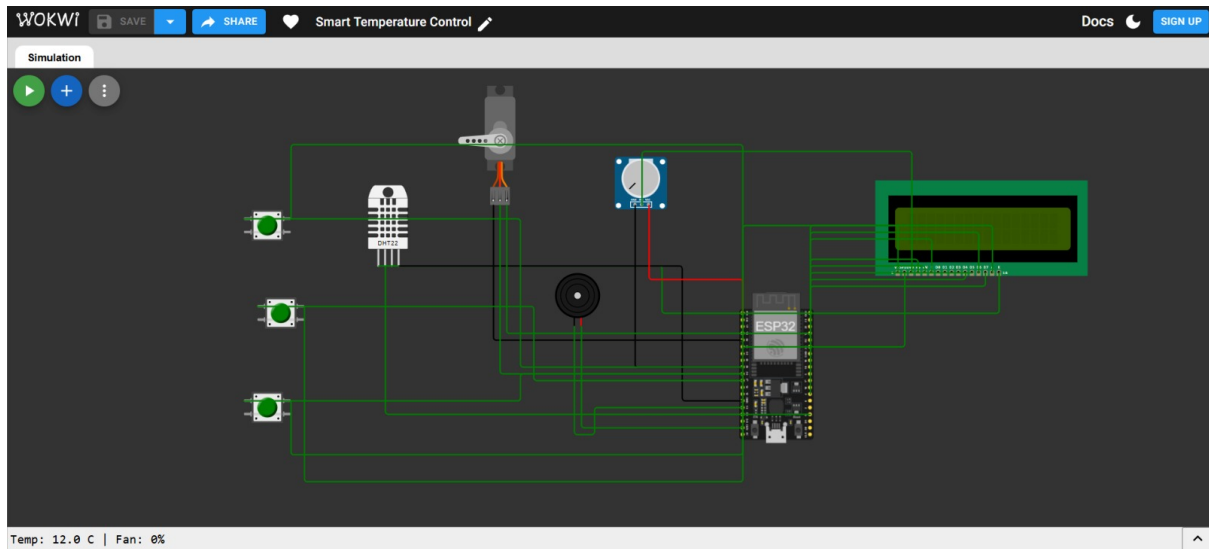


Figure 2: Wokwi Simulator — Full Circuit

5 Libraries Used

Library	Purpose
WiFi.h	Built-in ESP32 WiFi connectivity
ThingSpeak.h	Sending data to ThingSpeak cloud API
LiquidCrystal.h	Driving the 16x2 LCD in 4-bit mode
ESP32Servo.h	Controlling the servo motor on ESP32
DHTesp.h	Reading temperature from DHT22 sensor

6 What Was Implemented

Step 1 — Read Sensor Values

The DHT22 sensor is read every 2 seconds using the DHTesp library. If the sensor fails to return a valid reading, the system keeps the last known temperature, logs the failure to Serial, and shows `Sensor ERR!` on the LCD so you know something went wrong.

Step 2 — Connect to WiFi

On startup the ESP32 connects to Wokwi-GUEST WiFi. If the connection succeeds, the IP address is printed to Serial. If it doesn't connect within 10 seconds, the system continues running offline — local control still works, only uploads are skipped.

If WiFi drops after startup, the `uploadThingSpeak()` function automatically calls `WiFi.reconnect()` and waits up to 3 seconds before the next upload attempt. So a temporary drop doesn't break anything permanently.

Step 3 — Send Data to Cloud

Every 20 seconds, two values are sent to ThingSpeak:

- **Field 1** — Temperature in °C (from DHT22)
- **Field 2** — Fan speed as a percentage (0% to 100%)

The upload uses `ThingSpeak.writeFields()` which packages both fields into a single API call. A success prints `ThingSpeak: Upload OK` to Serial, and a failure prints the HTTP error code so you can debug it.

Step 4 — Visual Dashboard

ThingSpeak automatically creates a chart for Field 1 and Field 2. The dashboard at thingspeak.mathworks.com/channels/3423267 shows:

- A live line graph of temperature over time
- A live line graph of fan speed percentage over time
- Timestamps on every data point

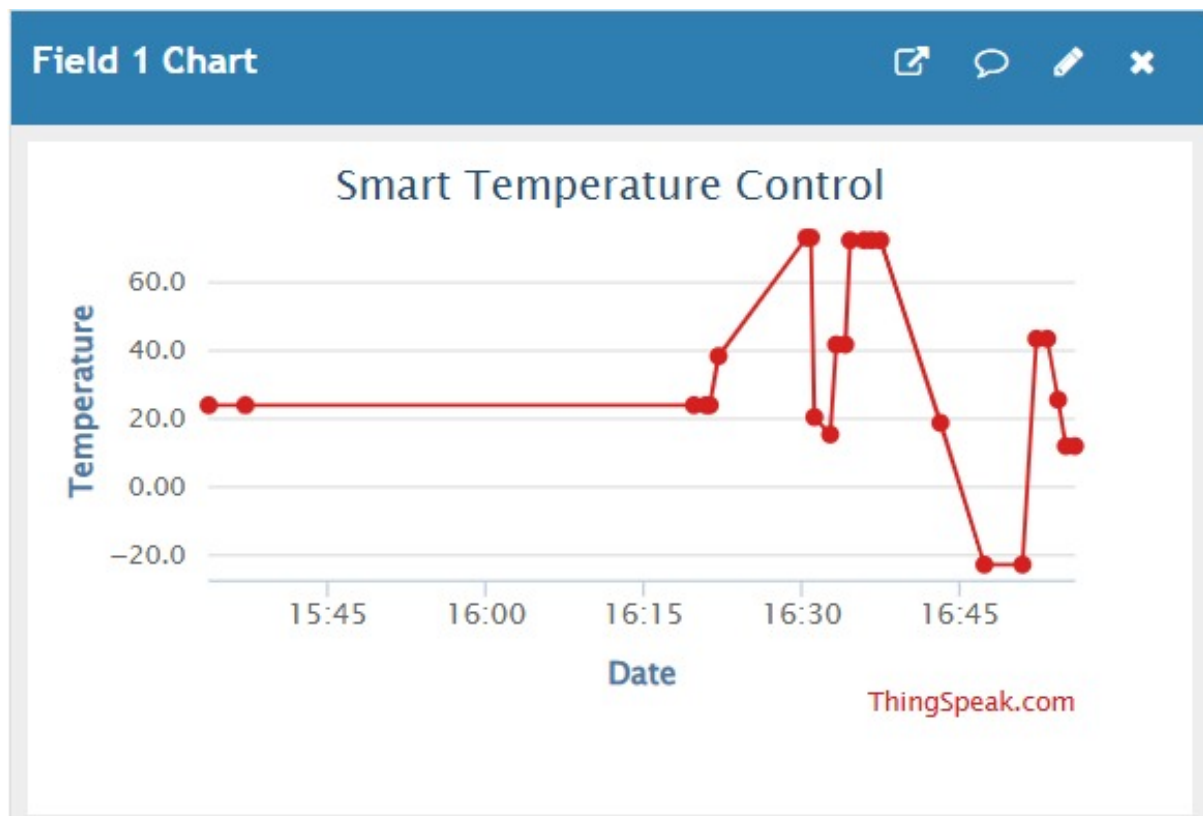


Figure 3: ThingSpeak — Field 1: Temperature (°C) over Time

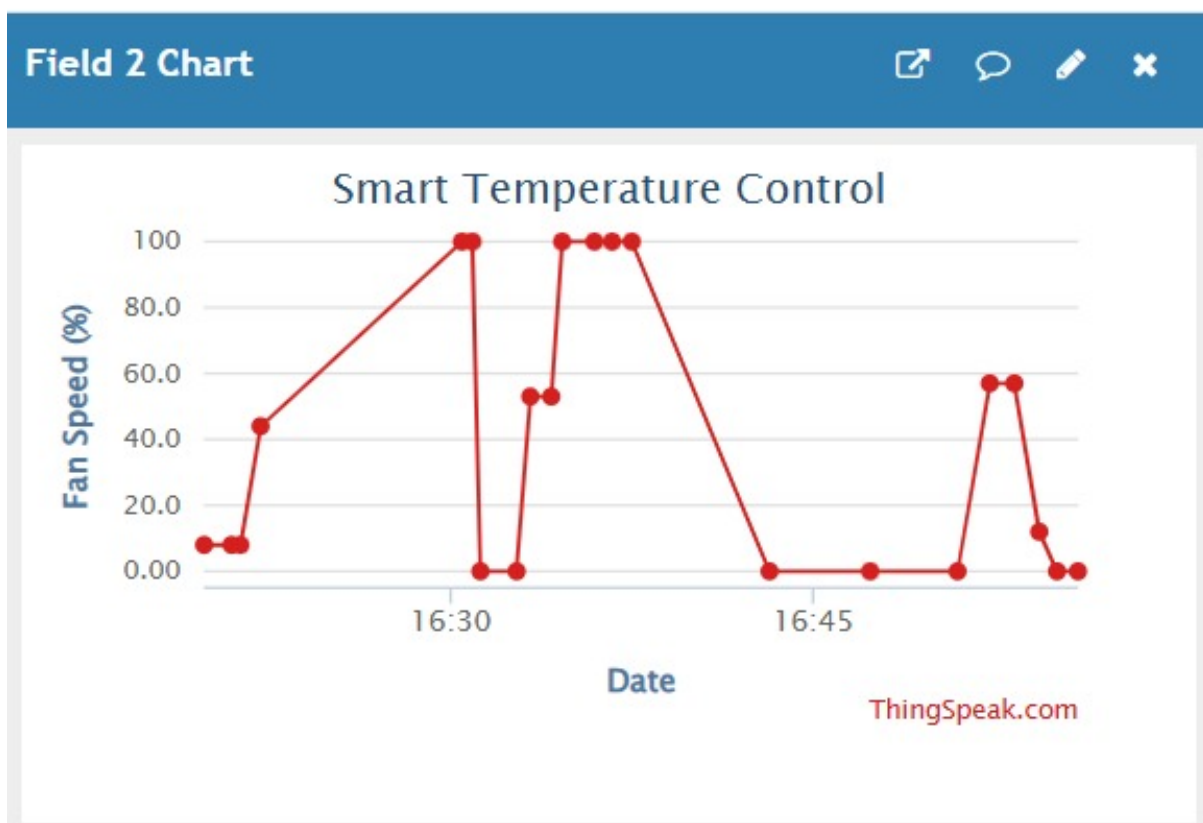


Figure 4: ThingSpeak — Field 2: Fan Speed (%) over Time

7 How the Device Sends Data to the Cloud

This is the part that actually makes it an IoT device rather than just a standalone controller. Here is the full flow, step by step:

1. The ESP32 boots up and connects to WiFi using its built-in radio. No extra module needed — WiFi is part of the chip.
2. Every 2 seconds, the DHT22 sensor gives a fresh temperature reading. The fan speed is calculated from that reading immediately.
3. Every 20 seconds, the `uploadThingSpeak()` function runs. It packages the current temperature and fan speed into two fields and calls `ThingSpeak.writeFields()`.
4. Internally, that function sends an HTTPS POST request to `api.thingspeak.com` with the channel ID and Write API Key as authentication. ThingSpeak verifies the key and stores the data point with a timestamp.
5. ThingSpeak's dashboard then renders that data point on the chart automatically. Anyone with the channel link can see the graph updating in near real-time from any browser, anywhere in the world.

The device never needs anyone to manually trigger an upload. It just runs, measures, and sends — continuously, on its own.

8 Full System Behaviour

Fan Control Logic

The fan (servo) speed is determined by two thresholds that the user can adjust using the physical buttons. The defaults are Low = 20.5°C and High = 60.5°C.

Temperature Zone	Servo Angle	Fan Speed
Below Low threshold	0°	0% (OFF)
Between Low and High	0° to 180° proportional	0% to 100%
Above High threshold	180°	100% (Full Speed)

Buzzer Alarm

If the temperature goes above the high threshold, `tone(BUZZER_PIN, 1000)` is called — this generates a 1 kHz square wave that the passive buzzer needs to actually make sound. When temperature drops back below the threshold, `noTone()` stops it immediately.

LCD Display

- **Startup:** Shows “Welcome To / Temp Control” for 2 seconds
- **Normal mode:** Row 1 shows temperature, Row 2 shows fan speed percentage
- **Setting mode:** Row 1 shows which threshold is being edited, Row 2 shows both current values

Button Behaviour

The SET button cycles through: Normal → Set Low Threshold → Set High Threshold → Normal. UP and DOWN adjust the selected threshold by 0.1°C per press. A guard prevents the Low threshold from ever meeting or crossing the High threshold, so the fan logic never breaks.

Each button fires exactly once per press using edge detection — the code detects the HIGH-to-LOW transition only after the signal has been stable for 50ms, and the button cannot fire again until it has been fully released and gone stable HIGH again. This fixed the continuous-repeat problem that came from the Wokwi bounce simulation on the SET button.

9 Conclusion

This project started as a basic temperature controller and ended up as a fully working IoT system that reads sensor data, controls hardware automatically, and uploads live readings to the cloud every 20 seconds — all running on an ESP32 inside the Wokwi simulator. The fan speed adjusts itself based on temperature, the buzzer triggers when things get too hot, the LCD shows what is happening locally, and ThingSpeak shows what is happening remotely. The buttons let the user change the thresholds without touching the code. Everything works together as one system. Along the way, real problems were solved — button bounce causing phantom presses, the passive buzzer staying silent because it needed `tone()` not `digitalWrite()`, WiFi dropping mid-session, and the display flickering from unnecessary `lcd.clear()` calls. These were not textbook problems, they came up during actual testing and got fixed one by one.